

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 - Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 - Articulation (BL3)	Assessment:	Assessment Tool 3 - Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	Niruthika Sri Prakash	Reg. No.:	192511294
Section / Batch:	2nd year / CSE	Date:	26/08/2026

Code of Conduct

I Niruthika Sri Prakash (192511294) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Niruthika Sri Prakash

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative Implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)

- Clustering effect
- Memory usage
- Performance under high load factor
- Which method is more suitable for large-scale applications and why?

2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)

3. Compare Quadratic Probing and Double Hashing in terms of:

- Clustering issues
- Collision resolution efficiency
- Suitability for dense hash tables

4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)

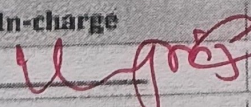
- Practical significance
- Impact on algorithm selection

5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)

- Space complexity (stack usage)
- Ease of implementation
- Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4 - Assessment Tool

Name: Nishuthika Sri Prakash

Regno: 192511294

Course Code: CSA0305

Course: Data Structures

①

Basis	Linear Probing	Separate chaining
Clustering effect	Suffers from Primary clustering, consecutive occupied position form cluster	Does not suffer from Primary clustering because collided elements are stored in buckets
memory usage	Uses only the hash table array so memory overhead is low.	Requires extra memory for linked lists/pointers used for each bucket.
Performance under high load factor	Performance decreases rapidly when load factor becomes high.	Performs comparatively better at high load factors.
Large Scale applications	Less Suitable when the table becomes highly loaded works well when load is low.	More Suitable for large-scale application because it handles collisions & high load factors.
Overall	Simple, cache-friendly and memory efficient but sensitive to clustering	Flexible & robust but uses additional memory.

(2)

Basics	Quick Sort	Merge Sort
Memory requirement for linked list	Can be implemented with low extra data space but recursion requires stack space.	Requires recursion. Stack, but merging linked lists can be done without extra array storage.
Time complexity	Average: $O(n \log n)$ Worst: $O(n^2)$	Best/Avg/Worst: $O(n \log n)$
In-Place	Can be in-place depending on implementation	For linked list, merging can be done by changing pointers.
Cache behavior	Usually good for arrays because elements are accessed close together. Poorer for linked lists.	Very suitable for linked lists because sequential traversal is used during merging.
10 million nodes	Risk of $O(n^2)$ in bad pivot cases and recursion overhead.	Predictable $O(n \log n)$ Performance & efficient pointer manipulation.
Linked list operations	Partitioning is more complicated because random access is unavailable.	Very efficient because splitting and merging can be performed using pointers.
For arrays	Quick Sort is often preferred because of good cache locality & low memory use.	Preferred when $O(n \log n)$ is stable.

Basics	Quadratic Probing	Double Hashing
clustering issues	Eliminates Primary clustering but can still suffer from Secondary clustering	Minimizes both Primary & Secondary clustering.
Probe Sequence	uses a Quadratic Sequence $h(k), h(k)+1^2, h(k)+2^2, \dots$	uses two hash functions: $h_1(k) + i \times h_2(k)$
Implementation	Relatively Simple implementation	Slightly more complex because, two good hash function
Dense hash tables	Less Suitable for very dense tables because finding empty positions becomes difficult.	More Suitable for dense hash tables due to better distribution.
Overall	Good improvement over linear Probing.	Generally provides better collision handling.

④

Algorithm	Best case	Worst case	Practical Significance
Bubble Sort	$O(n)$	$O(n^2)$	Good only for nearly sorted small data.
Insertion Sort	$O(n)$	$O(n^2)$	Very effective for small or nearly sorted data.
Selection Sort	$O(n^2)$	$O(n^2)$	Performance is almost independent of input order.
Quick Sort	$O(n \log n)$	$O(n^2)$	Usually very fast but poor Pivot Selection.
Merge Sort	$O(n \log n)$	$O(n \log n)$	Gives Predictable Performance for all input arrangement
Heap Sort	$O(n \log n)$	$O(n \log n)$	Provides guaranteed $O(n \log n)$ with low auxiliary space.

(5)

Basics	Recursive Quick Sort	Iterative Quick Sort
Space Complexity	Uses the recursion call Stack. Average $O(\log n)$	Uses explicit Stack. Average: $O(\log n)$ Worst: $O(n)$
Ease of implementation	Easy & Simple to implement. The recursive structure naturally matches quick sort.	More complex because Partition ranges must be stored & processed.
Performance	Generally efficient, but excessive recursion can cause problems.	Similar time complexity. Can be optimized to keep explicit stack size small.
Risk of Stack overflow	Higher risk, especially when poor pivot selection produces highly unbalanced partitions.	Lower risk of system call-stack overflow because programmer controls auxiliary stack.
Best use	Suitable for normal sized inputs and educational implementations.	Useful for very large inputs where avoiding recursion depth is important.
Stack usage	Managed automatically by the system.	Stack is managed manually using an stack data structure.

Uppof